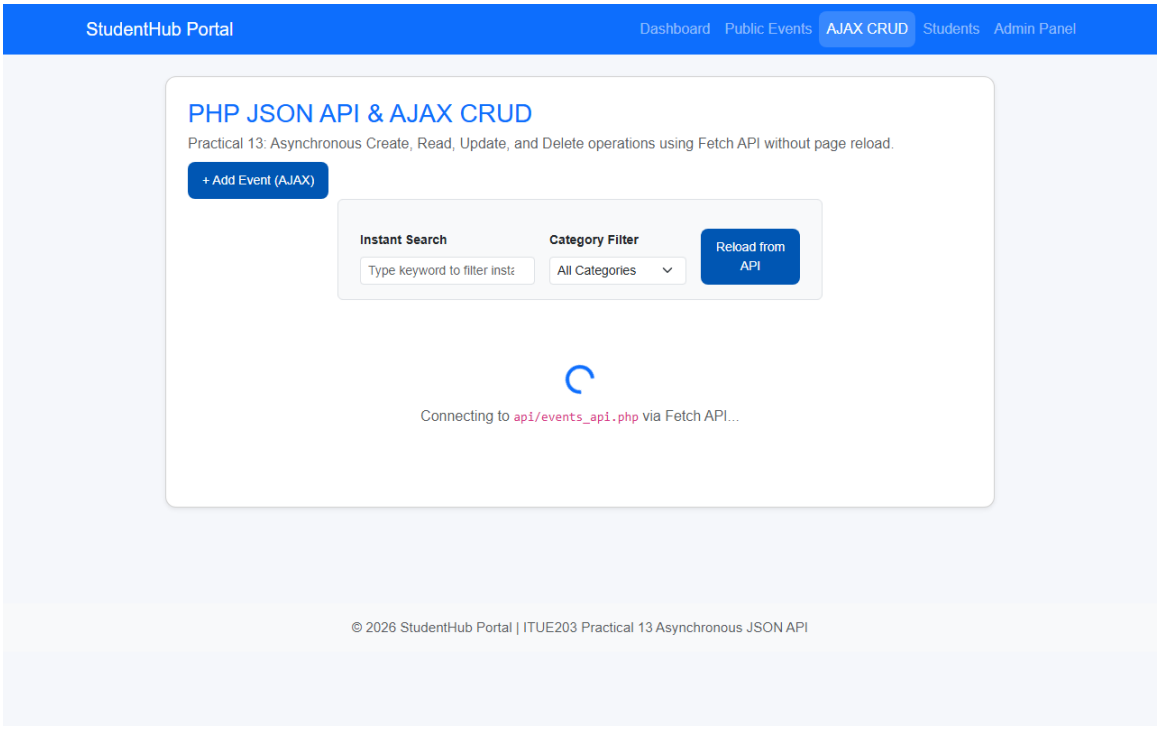
PRACTICAL – 13

Problem Definition: Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

Output:

AJAX Fetch CRUD Interface with Real-Time DOM Updates:



Question & Answer:

1. Do PHP endpoints return valid JSON responses?

Ans. Yes. The backend script api/events_api.php sets Content-Type: application/json; charset=utf-8 and outputs standardized JSON structures containing status codes, success booleans, messages, and payload arrays.

2. Are GET and POST requests handled properly?

Ans. Yes. The API endpoint routes requests based on the HTTP method: GET retrieves event lists or single records, POST creates new events, PUT updates existing events, and DELETE removes records.

3. Are loading, success, and error states shown on the frontend?

Ans. Yes. A loading spinner is displayed while Fetch API requests are pending. Toast notifications pop up upon successful completion of actions, and error banners alert the user if requests fail.

4. Is input validation performed before database operations?

Ans. Yes. Inputs are validated both on the client side before dispatching Fetch requests and strictly verified server side in PHP before database queries are executed.

Conclusion / Learning Outcome:

Practical 13 bridged frontend JavaScript and backend PHP through REST-style JSON APIs. Asynchronous CRUD interactions were implemented using the modern Fetch API, enabling seamless record creation, modification, deletion, and filtering without full page reloads.

Github link :- https://github.com/07vish/sh.git